tell me in detail as i will have to explain my teammates and maam, so tell me what tech stack is used, y is it used, is there any alternative then why not that, give me a flowchart of how this is working from start to end wid all users possible like admin, employee etc., also tell me where are the ml model files and how did you implement it, what is the use of both the models and how they work.
Here's a complete teammate/viva-ready walkthrough. I'd suggest reading it once, then keeping the ML + flowchart sections open during your viva as reference.

Part 1 — Tech stack, why each piece, and what we'd have used otherwise

The system has seven distinct layers. Each has an independent tech choice with tradeoffs we can defend.

1.1 Image preprocessing — **OpenCV + NumPy + Pillow**

Used for: deskew, denoise, binarise receipt images before OCR.

Why OpenCV:

- Battle-tested: used in every major computer-vision pipeline since 2000
- Pure-C under the hood → processes 626 receipts in ~3 minutes on a laptop
- Has every primitive we need (`cv2.minAreaRect`, `cv2.GaussianBlur`, `cv2.threshold` with Otsu)
- Works offline — no cloud dependency, no cost per image

Alternatives we considered:

Alternative	Why not
Pillow alone	Has basic image ops but no adaptive thresholding or Otsu; you'd end up reimplementing
scikit-image	Cleaner Python API but ~5× slower than OpenCV on the same operations
ImageMagick CLI	Great for one-off scripts, but shelling out per image would block the FastAPI request for s

1.2 OCR — **Google Gemini Flash 2.5 (Google AI Studio)**

Used for: extract vendor, date, amount, category, line items from the cleaned image → structured JSON.

Why Gemini Flash 2.5:

- Understands **context** — can infer "category: Food" from a menu of items, without being told. Classical OCR (Tesseract) can't do this.
- Handles handwritten amounts, unusual date formats, non-English words — SROIE has Malaysian receipts with both English and Malay
- Free tier usable for development (though limited per day)
- JSON-mode output with our schema-constrained prompt — no regex parsing of raw text

Alternatives we considered:

Alternative	Why not
Tesseract (open-source OCR)	Gives you raw text only. You'd then need a second step — entity extraction — wi
AWS Textract	Excellent, but paid from request one; also requires AWS IAM setup
Azure Document Intelligence	Similarly paid; locks you into Azure
GPT-4 Vision / Claude	Same capability but more expensive per call and stricter rate limits

1.3 Data preprocessing and ML — **scikit-learn + pandas + NumPy + imbalanced-learn**

Used for: feature engineering (11 numeric + 3 categorical features), train/test split, StandardScaler + OneHotEncoder, SMOTE resampling, Isolation Forest, Autoencoder.

Why scikit-learn:

- `Pipeline` + `ColumnTransformer` are the industry-standard way to make train/serve consistency foolproof — the same fitted object is used at both training and inference, so feature transformations can't drift.
- `joblib.dump` persists the fitted pipeline in a single file you load at inference in one line.
- Isolation Forest, MLPRegressor (our autoencoder), train_test_split with stratification — all built in.

Why imbalanced-learn:

- SMOTE (Synthetic Minority Over-sampling Technique) is the standard remedy for our 9 % fraud rate. It's maintained as a first-party sklearn-compatible package.

Alternatives we considered:

Alternative	Why not
TensorFlow / Keras for the autoencoder	The project spec nominates this. We use sklearn's `MLPRegressor` configured as an autoenco bottleneck → decoder), same MSE loss, same reconstruction-error scoring. Keras adds ~500 stack, is error-prone on Windows, and for a 29-dim input the neural net gains nothing from line change if a later requirement demands it.
PyTorch	Same story as Keras. More flexible but more code.
XGBoost anomaly	Supervised classifier — we'd need much more labelled fraud data to beat IF + AE.

1.4 Backend API — **FastAPI + uvicorn + pydantic**

Used for: expose `/api/submit`, `/api/companies/*`, `/api/config`, and serve the static frontend.

Why FastAPI:

- Async by default — when we call Gemini we don't block the worker; the spec's "Steps 3 + 4 in parallel via `asyncio.gather`" works naturally here
- Automatic request/response schema validation via pydantic — no manual JSON parsing
- Auto-generated OpenAPI docs at `/docs` if you want to show the API surface in your viva
- Tiny footprint — one file, one `app` object

Alternatives we considered:

Alternative	Why not
Flask	Synchronous by default; you'd bolt on `gevent` or rewrite for async. No built-in schema vali
Django / DRF	Batteries-included, but way too heavy for this scope. We'd spend more time on Django cor
Express / Node.js	Would work but then we'd need a Node ↔ Python bridge to reach our ML models — adds

1.5 Frontend — **Vanilla HTML + CSS + plain JavaScript (no framework)**

Used for: dashboard (`index.html`) and admin page (`admin.html`).

Why vanilla:

- Two pages, each <200 lines — a framework's boilerplate would be larger than the app
- Zero build step — you can edit files and just refresh
- `localStorage` handles the only state we needed to persist (phone number, tech-details toggle)
- Ideal for an FYP demo where simplicity reads as "I understand every line"

Alternatives we considered:

Alternative	Why not
React / Next.js (spec-mentioned)	The spec names these for a *production* deployment. For the FYP demo they'd add a bui of `node_modules`. The architecture stays valid — the backend API is the contract; swap
Vue / Svelte	Same argument.
Jinja2 templates (server-side render)	Would work but the interactive bits (reasons list, toggle, banners) are easier in JS.

1.6 Notifications — **Twilio Python SDK**

Used for: place a voice call to the uploader when the image is corrupt or OCR confidence is below 40 %.

Why Twilio: de-facto standard for programmable telephony; free trial gives $15 credit; voice + SMS in one SDK; inline TwiML means we don't need a hosted TwiML URL.

Alternatives: MessageBird, Plivo, AWS SNS (SMS only), Amazon Connect (heavier setup). Twilio has the widest free tier and best docs.

1.7 Persistence — **JSON files on disk**

Used for: company rules (`data/companies/*.json`), synthetic claims history (data/synthetic/claims.csv), trained models (`models/*.joblib`), processed features (`data/processed/*.parquet`).

Why files:

- Zero deployment friction — no DB to install, no migrations

- Human-readable — the admin page edits the same JSON you'd edit by hand
- Matches the spec exactly: *"Rules loaded from a config file — changing a policy limit doesn't require redeploying code."*

Alternatives: PostgreSQL + SQLAlchemy (the spec's Step 6 — Postgres is the intended production storage). For a viva-scale demo, files are clearer; the `CompanyStore` class has a `save_company` method that's trivially replaceable with an INSERT/UPDATE.

Part 2 — End-to-end flowchart with every actor

There are **two human actors** in the current system and one notification recipient:

- **Employee** — submits claims through the dashboard
- **Admin** (at each client company) — edits their company's rules via the admin page
- **(Notified employee)** — receives the Twilio call on a failed upload

2.1 Employee submits a claim (the main flow)

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│  EMPLOYEE                                                                            │
└──────────────────────────────────────────────────────────────────────────────────────┘
      │
      │  1. Opens http://127.0.0.1:8765  (frontend: index.html + app.js)
      │  2. Picks their Employee ID (dropdown auto-filled from /api/config)
      │     — Dept and Grade get auto-populated from synthetic data
      │     — Company ID is looked up from employee_map.json
      │  3. Uploads receipt.jpg
      │  4. (Optional) Fills override fields + phone number
      │  5. Clicks Submit
      ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│  FASTAPI  —  POST /api/submit                                                        │
│                                                                                      │
│              (backend/main.py, api_submit handler)                                   │
└──────────────────────────────────────────────────────────────────────────────────────┘
      │
      ├─▶ A. Image decode via OpenCV.
      │       • cv2.imdecode → numpy array
      │       • If it fails: return 400 + place Twilio call if phone given ─┐
      │                                                                     │
      ├─▶ B. Image preprocessing  (src/image_preprocessing/pipeline.py)     │
      │       • deskew()    — cv2.minAreaRect on text pixels                │
      │       • denoise()   — Gaussian blur + bilateral filter              │
      │       • binarise() — cv2.threshold with Otsu                        │
      │       → cleaned PNG bytes                                           │
      │                                                                     │
      ├─▶ C. Gemini Flash 2.5 OCR  (backend/ocr.py)                         │
      │       • POST image + schema-prompt to Google AI Studio              │
      │       • Parse the JSON out of the response                          │
      │       • If confidence < 0.4 + phone given: place Twilio call ───────┤
      │                                                                     │
```

```
├─▶ D. Resolve company  (backend/companies.py)
      • CompanyStore.company_for_employee(employee_id)
      • EMP00000-49 → ACME, EMP00050-99 → GLOBEX (map rule)
      • Load CompanyRules from data/companies/<id>.json
│
├─▶ E. Build claim record (stitching):
      { employee_id, department, grade,     ← metadata
        vendor, category, amount,           ← OCR (or overrides)
        submitted_at: datetime.now() }      ← server clock
│
├─▶ F. Feature engineering  (backend/features_online.py)
      • Use the FeatureStore (= synthetic claims.csv) to compute:
        amount_vs_category_mean, vendor_frequency, is_off_hours,
        vendor_repeat_count_3d, days_since_last_claim, …
      → 14-column dataframe (11 numeric + 3 categorical)
│
├─▶ G. Preprocessor transform (models/processed/preprocessor.joblib)
      • StandardScaler on numerics, OneHotEncoder on categoricals
      → 29-dim numeric vector
│
├─▶ H. Anomaly detection  (src/models/ensemble.py)
      • Isolation Forest → 0..1 anomaly score
      • Autoencoder → reconstruction error → 0..1 anomaly score
      • Combined = 0.5 × IF + 0.5 × AE
      • Top-3 features with highest reconstruction residual
│
├─▶ I. Classify verdict (backend/explain.py)
      • ML label: NORMAL / SUSPICIOUS / ANOMALOUS using company's
        own suspicious_threshold + anomalous_threshold
      • Generate all reasons: policy, behavioural, ml
      • final_verdict(): rules can ESCALATE the label but never
        downgrade it (spec's "rules override ML" principle)
│
└─▶ J. Response assembled and returned to browser. ──────────────────┐
                                                                      │
┌──────────────────────────────────────────────────────────────────────┐
│  BROWSER  (static/app.js)
│
│
│
│   • Renders 5 stages:
│
│      Stage 2: original + preprocessed images
│
│      Stage 3: Gemini OCR JSON
│
│      Stage 4: engineered features table
│
│      Stage 5: verdict panel + reasons + ML score bars
│
│   • Twilio notifications banner (if any calls placed)
│
│   • "Show technical details" toggle reveals z-scores etc.
└──────────────────────────────────────────────────────────────────────┘
                                                                      │
                                                                      ▼
┌──────────────────────────────────────────────────────────────────────┐
```

```
│   (NOTIFIED EMPLOYEE — only if Twilio call was placed)
│
│
│
│   Phone rings. Voice: "Hello. This is an automated notification from your
│
│   Expense Fraud demo. Your receipt upload failed…"
│
└──────────────────────────────────────────────────────────────────────────────
────┘
```

2.2 Admin edits company rules

```
┌──────────────────────────────────────────────────────────────────────────────
────┐
│   COMPANY ADMIN
│
└──────────────────────────────────────────────────────────────────────────────
────┘
      │
      │   1. Opens http://127.0.0.1:8765/admin
      │   2. Picks their company from the dropdown (ACME / GLOBEX)
      ▼
┌──────────────────────────────────────────────────────────────────────────────
────┐
│   GET /api/companies      → list of companies for the dropdown
│
│   GET /api/companies/:id  → current rules for the selected company
│
└──────────────────────────────────────────────────────────────────────────────
────┘
      │
      ▼
The admin UI renders 5 sections, one per rule type:
  1. Grade daily limits          — Junior / Mid / Senior / Director
  2. Per-category claim caps     — Food / Travel / Lodging / … (one per
category)
  3. Category restrictions       — chip grid: for each category, which
depts can claim it
  4. Round-number threshold      — the ₹ cutoff above which round amounts
flag
  5. Anomaly cutoffs             — SUSPICIOUS and ANOMALOUS score
boundaries
      │
      │   3. Admin changes values (e.g. ACME Food cap 500 → 300)
      │   4. Clicks "Save rules →"
      ▼
┌──────────────────────────────────────────────────────────────────────────────
────┐
│   PUT /api/companies/:id       (backend/main.py, api_update_company)
│
│
│
│   • CompanyStore.update_rules():
│
│       - merges with existing (partial update OK)
│
│       - _validate_rules() checks types, ranges, ordering
│
│       - atomic write: tmp file + rename
│
```

```
│       - in-memory cache updated
│
│   → returns the new CompanyRules JSON
│
└──────────────────────────────────────────────────────────────────────────────
──────┘
      │
      ▼
  From this moment on, every new /api/submit for an employee of THIS
company
  uses the new rules. No server restart. No retraining. No redeploy.
```

2.3 Manager review (spec'd in Step 8 but not implemented — you can mention as future work)

The project spec describes a third actor — a **Manager** — who reviews claims the system flagged as SUSPICIOUS and clicks Approve/Reject. We haven't built that page. The data model supports it: every `/api/submit` response has a full audit trail (claim, reasons, verdict) that a manager view could render. This is the natural next build.

Part 3 — ML model files and implementation

3.1 Where everything lives on disk

```
D:\fyp\
├── src\models\                          (source code for the models)
│   ├── isolation_forest_model.py        Wraps
sklearn.ensemble.IsolationForest
│   ├── autoencoder_model.py             Wraps
sklearn.neural_network.MLPRegressor
│   └── ensemble.py                      Combines IF + AE, scores claims,
serialises
│
├── scripts\train_models.py              Runs the training procedure,
writes artefacts
│
├── data\processed\
│   ├── features_train.parquet           1,760 rows × 29 features (pre-
SMOTE)
│   ├── features_train_resampled.parquet 3,200 rows × 29 features (SMOTE-
balanced)
│   ├── features_test.parquet            440 rows × 29 features (untouched)
│   └── preprocessor.joblib              Fitted StandardScaler +
OneHotEncoder
│
└── models\
    ├── isolation_forest.joblib          Trained IsolationForestScorer
    ├── autoencoder.joblib               Trained AutoencoderScorer
    ├── ensemble.joblib                  IF + AE bundled together (what the
API loads)
    └── metrics.json                     Test-set precision/recall/F1/AUC
```

3.2 How training ran — step by step

```
python scripts/train_models.py
```

This script does the following, in this exact order:

1. **Loads three splits** from `data/processed/`:

- Resampled train (for Isolation Forest — we want IF to see balanced class proportions)

- Original train (for the Autoencoder — we train it on **normal rows only**)
- Untouched test (for final evaluation)

2. **Trains Isolation Forest** on the resampled train set:

```
model = IsolationForest(n_estimators=200, contamination=0.1, n_jobs=-1)
model.fit(X_train_res)
```

Then records the min/max raw scores and picks a threshold at the 10th percentile — this is what makes the 0..1 normalisation work later.

3. **Trains Autoencoder** on **only the y=0 (normal) rows** from the original train:

```
model = MLPRegressor(hidden_layer_sizes=(16, 8, 16),
                     activation="relu", solver="adam",
                     max_iter=300, early_stopping=True)
model.fit(X_train_normal, X_train_normal)
```

Input and output are the same — it's learning to reconstruct normal claims. Records training reconstruction errors → used for 0..1 normalisation and for picking the threshold.

4. **Bundles** both scorers into an `Ensemble` object and evaluates on the untouched test set. Reports precision, recall, F1, ROC-AUC for each model and the combined ensemble. Writes everything to models/metrics.json.

3.3 What the training produced (your viva numbers)

From the actual models/metrics.json on your machine:

Model	Precision	Recall	F1	ROC-AUC
Isolation Forest alone	0.875	0.175	0.292	0.772
Autoencoder alone	0.423	0.550	0.478	0.822
Ensemble (IF + AE)	**0.900**	**0.450**	**0.600**	**0.806**

The viva story this tells:

- IF has the best precision — it rarely cries wolf, but misses most frauds
- AE has the best recall — it catches more frauds but has more false positives
- The ensemble beats **both** individual models on F1 (0.60 vs 0.48 vs 0.29) and gives the best precision of all three (0.90)
- This is exactly why the spec asks for the hybrid: each model covers the other's weakness

---

Part 4 — Why two models, not one, and how each actually works

4.1 Isolation Forest — the "fast obvious-outlier detector"

The one-line intuition: normal claims are hard to isolate from the crowd; fraud sticks out and can be isolated quickly.

How the algorithm builds a tree:

- Pick a random feature (say `amount`)
- Pick a random split value between its min and max
- Recurse on the two halves until every point is alone in its leaf

Do this 200 times (that's our `n_estimators=200`) and average the results.

How anomaly scoring works:

- A point's "isolation score" = average depth of the tree path to reach it
- **Shallower path = anomaly**. Think: a claim of ₹25,000 when the rest are ₹300–1,000 gets isolated in one or two splits
- **Deeper path = normal**. A typical claim sits in the middle of the feature distribution, needing many splits to separate

Our code then flips and rescales this to 0..1 so that 1 = most anomalous:

```
# src/models/isolation_forest_model.py line 43-48
raw = model.score_samples(X)          # higher raw = more normal
normed = (self.score_max - raw) / (self.score_max - self.score_min)
```

Strengths: fast, unsupervised, needs no fraud labels to train, handles high-dim data well.
Weaknesses: assumes anomalies are *far* from normal — it can miss subtle fraud that only deviates on one axis.

4.2 Autoencoder — the "pattern novelty detector"

The one-line intuition: train a neural net to compress-then-reconstruct normal claims. When a new claim arrives, if the net reconstructs it well, it's normal; if it fails, it's weird.

Architecture (29-dim input):

```
[input: 29 features]
        |
        ▼
┌──────────────┐
│ Dense(16)    │   ← encoder layer 1 (ReLU)
└──────────────┘
        |
        ▼
┌──────────────┐
│ Dense(8)     │   ← bottleneck (ReLU) — the compressed representation
└──────────────┘
        |
        ▼
┌──────────────┐
│ Dense(16)    │   ← decoder layer 1 (ReLU)
└──────────────┘
        |
        ▼
[output: 29 reconstructed features]
```

Training process:

- Only normal-class rows used (1,600 of them)
- Loss function: MSE between input and reconstruction

- Adam optimiser, early stopping when validation loss plateaus for 20 epochs
- Net learns a compact code of "what a normal claim looks like"

Inference scoring:

- For a new claim, run it through the net → get reconstruction
- Reconstruction error = mean squared difference between input and output
- Low error (≈0) → looks like what the net has seen → normal
- High error → the net's bottleneck couldn't compress this claim's information → anomaly

Our code:

```
# src/models/autoencoder_model.py line 64-67
reconstructed = self.model.predict(X)
errors = np.mean((X - reconstructed) ** 2, axis=1)
```

Strengths: excellent at subtle, multivariate anomalies — a claim that's "mostly normal but weird in two dimensions together" is caught.
Weaknesses: slower to train and score than IF; can miss obvious univariate outliers (a claim 100× the norm might reconstruct fine if only the amount is off).

4.3 Why combine them

Fraud pattern	IF c
₹25,000 claim when typical is ₹400 (obvious magnitude)	✓ s
Junior claiming Entertainment at 3 am on a weekend (subtle combination)	× o
Unfamiliar vendor + unfamiliar hour (two-feature correlation)	× m
Simple duplicate receipt (identical amount + vendor)	✓ s

By averaging their 0..1 scores with equal weight, we get a system that covers both kinds of signal and beats either model individually on F1 (as the metrics above show).

4.4 Explainability: the "top-3 residual features"

After the AE reconstructs a claim, we sort the per-feature residuals and take the top 3. These tell us **why** the AE thought the claim was weird. If `amount_vs_employee_mean` has the highest residual at 5.87 SDs above expected, it means "this employee spent way more than their personal norm" — which is exactly the kind of actionable reason that ends up on the dashboard.

Part 5 — Glossary of abbreviations you'll use in the viva

- **SROIE** — Scanned Receipts OCR and Information Extraction (ICDAR 2019 Task 3 dataset)
- **OCR** — Optical Character Recognition
- **IF** — Isolation Forest
- **AE** — Autoencoder
- **SMOTE** — Synthetic Minority Over-sampling TEchnique

- **E.164** — International phone number format (e.g. `+919876543210`)
- **TwiML** — Twilio Markup Language (XML telling Twilio what to say in a call)
- **SD** — Standard Deviation (units our ML residuals are expressed in)
- **TP/FP/TN/FN** — True/False Positive/Negative (confusion-matrix terms)
- **AUC / ROC-AUC** — Area under the Receiver Operating Characteristic curve (0.5 = random, 1.0 = perfect)